

■ 2.3.6 Advanced Topic: Functions with Variable Numbers of Arguments

When you make an assignment like `f[x_, y_] := value`, you are defining the value of the function `f` with two arguments. If you ask for values of `f` with different numbers of arguments, say `f[a]` or `f[a, b, c]`, *Mathematica* will not know their values.

You will sometimes need to define a function that can take a variable number of arguments. While a “single blank” `_` stands for a single *Mathematica* expression, you can use a “double blank” `__` to stand for any *sequence* of expressions.

This defines a value for the function `f` with exactly two arguments.

```
In[1]:= f[x_, y_] := x + y
```

No values are defined for `f` with one or three arguments.

```
In[2]:= f[a] + f[a, b, c]
Out[2]= f[a] + f[a, b, c]
```

This defines the function `g` with any sequence of (one or more) arguments.

```
In[3]:= g[x_] := p[x] + q[x, x]
```

The definition given for `g` applies, with `x` taken as the sequence `(a, b, c)`.

```
In[4]:= g[a, b, c]
Out[4]= p[a, b, c] + q[a, b, c, a, b, c]
```

Here is a more complicated definition, which picks out pairs of duplicated elements in `h`.

```
In[5]:= h[a___, x_, b___, x_, c___] := hh[x] h[a, b, c]
```

The rule is applied twice, picking out the two paired elements.

```
In[6]:= h[2, 3, 2, 4, 5, 3]
Out[6]= h[4, 5] hh[2] hh[3]
```

“Double blanks” `__` stand for sequences of one or more expressions. “Triple blanks” `___` stand for sequences of zero or more expressions. You should be very careful whenever you use triple blank patterns. It is easy to make a mistake that can lead to an infinite loop. For example, if you define `p[x_, y___] := p[x] q[y]`, then typing in `p[a]` will lead to an infinite loop, with `y` repeatedly matching a sequence with zero elements. Unless you are sure you want to include the case of zero elements, you should always use double blanks rather than triple blanks.

-	any single expression
$x_$	any single expression, to be named x
$_$	any sequence of one or more expressions
$x_$	sequence named x
x_h	sequence of expressions, all of whose heads are h
$x_$	sequence of zero or more expressions

More kinds of pattern objects.

Notice that when you work with “flat” functions such as **Plus** and **Times**, variable numbers of arguments are automatically handled, without you explicitly needing to use double or triple blanks. Section 2.2.18 discusses how you can set up your own “flat” functions.

Sometimes you will want to define a function where certain arguments, if omitted, should be given “default values”. The pattern $x_:v$ stands for an object that can be omitted, and replaced by the default value v .

This defines a function j with a required argument x , and optional arguments y and z , with default values 1 and 2, respectively.

```
In[7]:= j[x_, y_:1, z_:2] := jp[x, y, z]
```

The default value of z is used here.

```
In[8]:= j[a, b]
Out[8]= jp[a, b, 2]
```

Now the default values of both y and z are used.

```
In[9]:= j[a]
Out[9]= jp[a, 1, 2]
```

Some common functions, such as **Plus**, have built-in default values for their arguments. In these cases, you can use the special notation $x_.$ to stand for an argument that can be omitted. In the case of arguments to **Plus**, $x_.$ is in fact equivalent to $x_:0$. The notation $x_.$ has the obvious advantage, however, that you do not need to specify the default value explicitly every time you set up a pattern. Section 2.3.7 will discuss how you can set up your own “default defaults” for arguments in functions.

$x_:v$	an expression which, if omitted, is taken to have default value v
$x_.$	an expression with a built-in default value

Pattern objects with default values.